

1. Goal

This exercise set is designed for the moment right after Lecture 4, after you have implemented HW1 DDPM and seen that the score parameterization performed much worse than epsilon and velocity prediction.

The target is not to memorize more formulas. The target is to be able to explain:

- what the score $\nabla_x \log p_{t(x)}$ means,
- how DDPM epsilon prediction is related to score prediction,
- why mathematically equivalent parameterizations can train very differently,
- how this prepares you for HW2 Flow Matching.

Suggested time: 75–100 minutes.

2. Part A: Rebuild the DDPM-to-Score Bridge

2.1. A1. Forward noising formula

Start from the closed-form DDPM forward process:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Write down the conditional distribution $q(x_t \mid x_0)$.

🔖 Small theorem: affine transformation of a Gaussian

If $Z \sim \mathcal{N}(\mu, \Sigma)$ and $Y = AZ + b$, then

$$Y \sim \mathcal{N}(A\mu + b, A\Sigma A^T).$$

For the scalar-scale case $Y = cZ + b$, this becomes

$$Y \sim \mathcal{N}(c\mu + b, c^2\Sigma).$$

The important point is that shifting by b changes the mean, while scaling by c changes the covariance by c^2 .

🔖 Reminder: expectation and covariance

For a random vector Y , its mean is

$$\mu_Y = \mathbb{E}[Y].$$

Covariance measures the average outer product of the deviation from the mean:

$$\text{Cov}(Y) = \mathbb{E}[(Y - \mathbb{E}[Y])(Y - \mathbb{E}[Y])^T].$$

This is the vector version of variance. If $Y = (Y_1, \dots, Y_d)$, then the covariance matrix entry at row i , column j is

$$\text{Cov}(Y)_{ij} = \mathbb{E}[(Y_i - \mathbb{E}[Y_i])(Y_j - \mathbb{E}[Y_j])].$$

The diagonal entries are variances, and the off-diagonal entries describe how two coordinates move together:

$$\text{Cov}(Y)_{ii} = \text{Var}(Y_i).$$

In this exercise, the image tensor can be thought of as one large vector. The covariance term in a Gaussian distribution tells us how much random noise is added around the mean image. The identity matrix I means the noise is isotropic: each coordinate has the same variance, and different coordinates have zero cross-covariance.

Your work

Given x_0 , the clean sample is fixed. The only randomness in x_t comes from ε . The DDPM forward process is

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Match this to the affine-Gaussian rule $Y = cZ + b$:

$$Z = \varepsilon, \quad c = \sqrt{1 - \bar{\alpha}_t}, \quad b = \sqrt{\bar{\alpha}_t}x_0.$$

Since ε is Gaussian, $x_t \mid x_0$ is also Gaussian. The conditioning matters here: after conditioning on x_0 , the term involving x_0 is treated as a fixed shift, while only ε remains random. Its conditional mean is

$$\mathbb{E}[x_t \mid x_0] = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\mathbb{E}[\varepsilon] = \sqrt{\bar{\alpha}_t}x_0.$$

Its conditional covariance is

$$\text{Cov}(x_t \mid x_0) = \text{Cov}(\sqrt{1 - \bar{\alpha}_t}\varepsilon) = (\sqrt{1 - \bar{\alpha}_t})^2 \text{Cov}(\varepsilon) = (1 - \bar{\alpha}_t)I.$$

The fixed shift $\sqrt{\bar{\alpha}_t}x_0$ changes the center of the Gaussian but not its spread, so it appears in the mean but drops out of the covariance.

Therefore,

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

Now derive the conditional score with respect to x_t :

$$\nabla_{x_t} \log q(x_t \mid x_0).$$

Hint: for $x \sim \mathcal{N}(\mu, \sigma^2 I)$,

$$\nabla_x \log p(x) = -\frac{x - \mu}{\sigma^2}.$$

Your derivation

From the previous part,

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

Compare this with the generic isotropic Gaussian $x \sim \mathcal{N}(\mu, \sigma^2 I)$. In this conditional distribution,

$$x = x_t, \quad \mu = \sqrt{\bar{\alpha}_t}x_0, \quad \sigma^2 = 1 - \bar{\alpha}_t.$$

The hint uses $p(x)$ as a generic density name. Here the concrete density is $q(x_t \mid x_0)$, so we substitute the conditional mean and variance into the Gaussian score formula:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1 - \bar{\alpha}_t}.$$

Now use the forward noising equation

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon.$$

Therefore,

$$x_t - \sqrt{\bar{\alpha}_t}x_0 = \sqrt{1 - \bar{\alpha}_t}\varepsilon.$$

Substitute this into the score:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{\sqrt{1 - \bar{\alpha}_t}\varepsilon}{1 - \bar{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}.$$

Expected destination

Expected destination:

$$\nabla_{x_t} \log q(x_t | x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1 - \bar{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}.$$

2.2. A2. Convert between prediction targets

For each model output below, derive the equivalent predicted noise $\hat{\varepsilon}_\theta$.

Target	Model output	Derive $\hat{\varepsilon}_\theta$
ε	$\hat{\varepsilon}_{\theta(x_t, t)}$	$\hat{\varepsilon}_\theta = \hat{\varepsilon}_{\theta(x_t, t)}$
x_0	$\hat{x}_{0, \theta}(x_t, t)$	$\hat{\varepsilon}_\theta = \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_{0, \theta}}{\sqrt{1 - \bar{\alpha}_t}}$
score	$\hat{s}_{\theta(x_t, t)}$	$\hat{\varepsilon}_\theta = -\sqrt{1 - \bar{\alpha}_t} \hat{s}_\theta$
velocity	$\hat{v}_{\theta(x_t, t)}$	$\hat{\varepsilon}_\theta = \sqrt{1 - \bar{\alpha}_t} x_t + \sqrt{\bar{\alpha}_t} \hat{v}_\theta$

Write the velocity definition you are using before deriving the inverse.

What this table is doing

The model always receives the same kind of input, x_t and t , but we can train it to output different targets: noise, clean image, score, or velocity. A hat means a model estimate, so $\hat{\varepsilon}_\theta$ is the model-estimated noise. This table converts each possible raw model output back into a common predicted-noise form, because the DDPM sampling equation is easiest to write using $\hat{\varepsilon}_\theta$.

Your derivation

Start from the forward noising equation:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon.$$

1. Epsilon prediction. If the model already outputs noise, no conversion is needed:

$$\hat{\varepsilon}_\theta = \hat{\varepsilon}_{\theta(x_t, t)}.$$

2. Clean-image prediction. If the model outputs $\hat{x}_{0, \theta}$, substitute this estimate for x_0 and solve the forward equation for ε :

$$x_t = \sqrt{\bar{\alpha}_t}\hat{x}_{0, \theta} + \sqrt{1 - \bar{\alpha}_t}\hat{\varepsilon}_\theta.$$

Therefore,

$$\hat{\varepsilon}_\theta = \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_{0, \theta}}{\sqrt{1 - \bar{\alpha}_t}}.$$

3. Score prediction. From A1, the conditional score is a rescaled noise target:

$$s_t = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}.$$

If the model outputs $\hat{s}_\theta$, invert this relation:

$$\hat{\varepsilon}_\theta = -\sqrt{1 - \bar{\alpha}_t}\hat{s}_\theta.$$

4. Velocity prediction. Use the HW1/repo convention

$$v_t = \sqrt{\bar{\alpha}_t}\varepsilon - \sqrt{1 - \bar{\alpha}_t}x_0.$$

Let $a = \sqrt{\bar{\alpha}_t}$ and $b = \sqrt{1 - \bar{\alpha}_t}$. Then the two equations are

$$x_t = ax_0 + b\varepsilon, \quad v_t = a\varepsilon - bx_0.$$

To isolate ε , compute

$$bx_t + av_t = b(ax_0 + b\varepsilon) + a(a\varepsilon - bx_0) = abx_0 + b^2\varepsilon + a^2\varepsilon - abx_0 = (a^2 + b^2)\varepsilon.$$

Since $a^2 + b^2 = \bar{\alpha}_t + (1 - \bar{\alpha}_t) = 1$, we get

$$\varepsilon = bx_t + av_t.$$

Therefore the predicted-noise conversion is

$$\hat{\varepsilon}_\theta = \sqrt{1 - \bar{\alpha}_t}x_t + \sqrt{\bar{\alpha}_t}\hat{v}_\theta.$$

2.3. A3. Explain equivalent but not equal

In HW1, all four prediction targets can be converted back to an epsilon prediction for sampling. Yet their KID scores were very different.

Write a short explanation in three sentences:

1. One sentence about mathematical equivalence.
2. One sentence about optimization target scale.
3. One sentence about why KID/noise MSE are more comparable than raw training loss.

Your explanation

The four parameterizations are mathematically equivalent at sampling time because each model output can be converted into a common predicted-noise quantity $\hat{\varepsilon}_\theta$ and then used in the same DDPM reverse update. However, they are not optimization-equivalent: the network is trained against different target tensors with different scales and timestep dependence, so the raw MSE losses can have very different magnitudes and difficulty even when the targets encode related information. Therefore, converted noise MSE and image-level metrics such as KID are more comparable than raw training loss, because they evaluate the models after mapping them back to a shared sampling quantity or final sample quality.

3. Part B: Numerical Scale Check

The score target used in your HW1 implementation is:

$$s_t = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}.$$

This means the score target is a rescaled version of the same Gaussian noise. The rescaling depends strongly on t .

3.1. B1. Compute the scale by hand

Using the HW1 schedule $\beta_1 = 0.0001$ and $\bar{\alpha}_1 \approx 1 - \beta_1$, estimate:

$$\frac{1}{\sqrt{1 - \bar{\alpha}_1}}.$$

Your estimate

Using $\bar{\alpha}_1 \approx 1 - \beta_1$ with $\beta_1 = 0.0001$:

$$\bar{\alpha}_1 \approx 0.9999, \quad 1 - \bar{\alpha}_1 \approx 0.0001.$$

Therefore,

$$\frac{1}{\sqrt{1 - \bar{\alpha}_1}} = \frac{1}{\sqrt{0.0001}} = \frac{1}{0.01} = 100.$$

What does this number mean for the target magnitude at very small timestep?

Interpretation

At a very small timestep, $\bar{\alpha}_t$ is close to 1, so $1 - \bar{\alpha}_t$ is tiny. Since the score target is

$$s_t = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}},$$

the same noise ε is multiplied by a large scale factor. For $t = 1$ under this schedule, the factor is about 100, so the score target can be roughly two orders of magnitude larger than the original noise target.

This matters for training because an MSE loss on score prediction is measuring errors in this enlarged target space. Even a modest error in the underlying noise direction can become a much larger raw score-space loss at early timesteps, so the score loss scale is not directly comparable to the epsilon loss scale.

3.2. B2. Tiny Python check

Run this snippet locally or in a notebook. It does not need PyTorch.

```
1 import math
2
3 T = 1000
4 beta_start = 1e-4
5 beta_end = 0.02
6
7 alpha_bar = 1.0
8 rows = []
9 for t in range(T):
10     beta_t = beta_start + (beta_end - beta_start) * t / (T - 1)
11     alpha_bar *= 1.0 - beta_t
12     sigma_t = math.sqrt(1.0 - alpha_bar)
13     rows.append((t, alpha_bar, sigma_t, 1.0 / sigma_t))
14
15 for t in [0, 1, 9, 99, 499, 999]:
16     _, alpha_bar_t, sigma_t, score_scale = rows[t]
17     print(t, alpha_bar_t, sigma_t, score_scale)
```

Record the scale at $t = 0$, $t = 99$, and $t = 999$.

t	$\sqrt{1 - \bar{\alpha}_t}$	$\frac{1}{\sqrt{1 - \bar{\alpha}_t}}$
0	0.010000	100.000000
99	0.320908	3.116159
999	0.999980	1.000020

3.3. B3. Interpret the result

Answer:

- Why can the score loss be much larger than the epsilon loss?
- Why does a large score loss not automatically mean the model is useless?
- Why can the final generated KID still be bad even if converted noise MSE is only moderately worse?

Your answer

B2 shows that the score target scale $\frac{1}{\sqrt{1-\bar{\alpha}_t}}$ is highly timestep-dependent: it is about 100 at $t = 0$, about 3.12 at $t = 99$, and about 1 at $t = 999$. Since the score target is

$$s_t = -\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}},$$

an error measured in score space can be much larger than the corresponding error measured in epsilon space, especially at early low-noise timesteps. This is why the raw score MSE loss can be much larger than the raw epsilon MSE loss: the target itself has been rescaled by a large factor.

However, a large raw score loss does not automatically mean the model is useless. The score prediction can be converted back to a noise prediction by

$$\hat{\varepsilon}_\theta = -\sqrt{1-\bar{\alpha}_t}\hat{s}_\theta.$$

After this conversion, the effective noise prediction error may be less dramatic than the raw score-space loss suggests. Therefore, raw losses across epsilon, score, x_0 , and velocity parameterizations are not directly comparable unless their target scales are taken into account.

At the same time, final KID can still be bad even if the converted noise MSE is only moderately worse. Sampling is an iterative reverse process, so small or moderate errors in predicted noise can accumulate across many denoising steps and affect final image quality. KID measures the distribution of generated images rather than one-step target error, so it can reveal sample-quality problems that are not fully captured by converted noise MSE alone.

4. Part C: Code-Reading Exercise

Use the actual HW1 implementation.

4.1. C1. Locate the target formulas

Open `src/methods/ddpm.py` and find `_get_training_target`.

For each branch, write the target tensor:

prediction_type	training target
epsilon	ε / noise
x_0	x_0
v	$\sqrt{\bar{\alpha}_t}\varepsilon - \sqrt{1-\bar{\alpha}_t}x_0$
score	$-\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}}$

4.2. C2. Locate the sampling conversion

Find `_prediction_to_noise`.

Question: why is it useful to convert every prediction type back to epsilon before reverse sampling?

Your answer

The DDPM reverse sampling equation is easiest to implement and check when it is written in terms of a predicted noise $\hat{\varepsilon}_\theta$. Different training parameterizations produce different raw model outputs, but `_prediction_to_noise` maps all of them into the same common object: an epsilon prediction. This keeps the reverse process shared across parameterizations instead of writing a separate sampler for `epsilon`, `x0`, `v`, and `score`.

Now answer the sharper version:

If score sampling converts back to epsilon, why can the score model still produce much worse samples?

Your answer

The conversion is algebraically correct, but it does not undo optimization difficulty. The score target is $-\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$, so its scale changes strongly with timestep and can become very large at early timesteps. If the model learns this rescaled target less accurately, then converting back to epsilon still gives a noisier or biased $\hat{\epsilon}_\theta$. During iterative sampling, those errors can accumulate across many reverse steps and produce worse final samples.

4.3. C3. One-line debug metric

Suppose you are debugging a bad score model. Which metric is more useful for comparing with the epsilon model?

- raw `loss`
- converted `noise_mse`
- image KID
- timestep-bin `noise_mse`

Rank them from most immediately diagnostic to least immediately diagnostic, and explain your ranking.

Your ranking

Most immediately diagnostic: timestep-bin `noise_mse`, then converted `noise_mse`, then image KID, then raw `loss`. Timestep-bin `noise_mse` is the most useful first check because the score target scale is highly timestep-dependent; binning shows whether the score model fails especially at early, middle, or late timesteps after converting back to epsilon. Converted `noise_mse` is the next best metric because it compares the score model and epsilon model in the same predicted-noise space. Image KID is important for final sample quality, but it is less immediate for debugging because many sampling-step errors are collapsed into one image-level distribution score. Raw `loss` is least comparable across parameterizations because each target has a different scale, so a larger raw score loss may reflect target scaling rather than strictly worse denoising ability.

5. Part D: Mini Coding Task

This is a small coding exercise, not a repo change requirement. Put it in a scratch notebook or temporary script.

5.1. D1. Score target visualizer

Write a function:

```
1 def score_scale_table(
2     num_timesteps: int = 1000,
3     beta_start: float = 1e-4,
4     beta_end: float = 0.02,
5 ) -> list[tuple[int, float, float]]:
6     """Return (t, sqrt_one_minus_alpha_bar_t, score_scale_t)."""
```

Then print a compact table for timesteps `[0, 1, 9, 99, 499, 999]`.

Before coding, write the shape-free formula you are implementing:

Formula

For a linear beta schedule,

$$\beta_t = \beta_{\text{start}} + (\beta_{\text{end}} - \beta_{\text{start}}) \frac{t}{T-1}, \quad \alpha_t = 1 - \beta_t.$$

The cumulative signal retention is

$$\bar{\alpha}_t = \prod_{s=0}^t \alpha_s.$$

The score target scale is

$$s_t = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}, \quad \text{scale}_t = \frac{1}{\sqrt{1 - \bar{\alpha}_t}}.$$

So the script computes and prints

$$\left(t, \sqrt{1 - \bar{\alpha}_t}, \frac{1}{\sqrt{1 - \bar{\alpha}_t}} \right).$$

After coding, paste your output:

Output

Running `python3 exercise/lecture-4-score/score_scale_table.py` gives:

t	sqrt(1-alpha_bar_t)	1/sqrt(1-alpha_bar_t)
0	0.0100000000	100.0000000000
1	0.0148292929	67.4340986257
9	0.0435292455	22.9730607185
99	0.3209078596	3.1161592657
499	0.9599024727	1.0417725013
999	0.9999798206	1.0000201798

5.2. D2. Optional PyTorch version

If your environment has PyTorch, create a tiny batch:

```
1 x0 = torch.randn(4, 3, 8, 8)
2 t = torch.tensor([0, 9, 99, 999])
3 noise = torch.randn_like(x0)
```

Build the score target and check:

$$-\sqrt{1 - \bar{\alpha}_t} \cdot s_t = \varepsilon.$$

What should the maximum absolute error be, up to floating point precision?

Your answer

The check should recover ε from the score target:

$$s_t = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}.$$

Multiplying by $-\sqrt{1 - \bar{\alpha}_t}$ gives

$$-\sqrt{1 - \bar{\alpha}_t} s_t = \varepsilon.$$

Therefore the maximum absolute error should be approximately 0, up to floating point precision. In practice, a small value such as 10^{-7} to 10^{-6} would be expected for float32 computations.

Local note: PyTorch was not available in the current shell environment when this handout was edited, so this optional check is written as the expected result rather than a recorded run.

6. Part E: Bridge to HW2

Lecture 4 closes one loop: DDPM can be interpreted through score/noise prediction. HW2 opens the next loop: Flow Matching trains a velocity field instead of a denoising score.

Answer these before starting HW2 implementation.

6.1. E1. Compare the learned object

Fill this table.

Method	What the network predicts	How sampling moves from noise to data
DDPM epsilon	Noise ε added in the forward process, usually written $\hat{\varepsilon}_{\theta(x_t, t)}$	Start from Gaussian noise and repeatedly apply learned reverse denoising steps, using $\hat{\varepsilon}_{\theta}$ to estimate the mean of $p_{\theta(x_{t-1} x_t)}$
Score model	Score field $\nabla_x \log p_{t(x)}$, or in this DDPM conditional case $-\frac{\varepsilon}{\sqrt{1-\alpha_t}}$	Use the score as a direction that points toward higher data density; sampling can be done by Langevin dynamics, annealed Langevin dynamics, or reverse-time SDE-style updates
Flow Matching	Velocity/vector field $v_{\theta(x, t)}$ that transports samples along a path from noise/source distribution to data distribution	Start from noise and integrate the learned vector field, e.g. with Euler steps, so samples move continuously from source to data

6.2. E2. Your readiness check

You are ready to start HW2 Part I if you can say these out loud without looking:

- DDPM forward process formula.
- Score of $q(x_t | x_0)$ and why it rescales epsilon.
- Why raw losses are not comparable across parameterizations.
- What a velocity/vector field means at a high level.
- Why fewer-step sampling is central to HW2.

For each item, mark:

Concept	Can explain	Half clear	Need review
Forward process	✓		
Score/equivalent epsilon	✓		
Loss comparability	✓		
Velocity field		✓	
Sampling steps		✓	

7. Suggested Solution Sketches

Do this section after attempting the exercises.

7.1. A1

Since

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I),$$

the conditional score is:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1 - \bar{\alpha}_t}.$$

Using

$$x_t - \sqrt{\bar{\alpha}_t}x_0 = \sqrt{1 - \bar{\alpha}_t}\varepsilon,$$

we get:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}}.$$

7.2. A2

ε

prediction is already $\hat{\varepsilon}_\theta$.

For x_0 prediction:

$$\hat{\varepsilon}_\theta = \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_{0,\theta}}{\sqrt{1 - \bar{\alpha}_t}}.$$

For score prediction:

$$\hat{\varepsilon}_\theta = -\sqrt{1 - \bar{\alpha}_t}\hat{s}_\theta.$$

For velocity prediction with

$$v_t = \sqrt{\bar{\alpha}_t}\varepsilon - \sqrt{1 - \bar{\alpha}_t}x_0,$$

the inverse is:

$$\hat{\varepsilon}_\theta = \sqrt{1 - \bar{\alpha}_t}x_t + \sqrt{\bar{\alpha}_t}\hat{v}_\theta.$$

7.3. B interpretation

The score target is a timestep-dependent rescaling of epsilon. At early low-noise timesteps, $\sqrt{1 - \bar{\alpha}_t}$ is tiny, so dividing by it makes the target large. That changes optimization even though the target can be converted back to epsilon exactly in algebra.

For the HW1 result, the clean explanation is: score prediction is mathematically valid, but the simple homework recipe does not normalize or reweight the score target, so it is a harder training problem than epsilon or velocity prediction.

7.4. E bridge

DDPM learns a denoising-related object and then repeatedly applies a reverse diffusion update. Flow Matching learns a vector field that moves samples along a continuous path from noise to data. This is why HW2 will care about Euler integration and the number of sampling steps.